JSPM's
Rajarshi Shahu College of Engineering,
Tathawade, Pune – 33.
Department of Computer Science and Business Systems
AY: 2026-27 Sem: III

Mini Project Report On

DAILY EXPENSE TRACKER

Submitted by:

Aayush Kumar (RBT25CB045)

Ritesh Buchade (RBT25CB049)

Under the guidance of

Prof. Rutuja S. Bachhav

Course: DBMS

Academic Year: **2026–27 (Semester III)**

JSPM's
Rajarshi Shahu College of Engineering,
Tathawade, Pune – 33.
Department of Computer Science and Business Systems
AY: 2026-27 Sem: III

Table of Contents

1. Course Outcomes 3
2. PO Mapped 4
3. PSO Mapped 5
4. SDG Goals Mapped 6
5. Abstract 7
6. Introduction 8
6.1 Background and Motivation 8
6.2 Problem Definition 8
6.3 Objectives 8
7. Literature Survey / Existing System 9
7.1 Existing Ways of Tracking Expenses 9
7.2 Limitations of Manual Methods 9
7.3 Gaps Identified 9
8. Proposed System 10
8.1 System Overview 10
8.2 Features and Scope 10
8.3 System Architecture Diagram 10
9. Methodology / Implementation 12
9.1 Tools and Technologies Used 12
9.2 Modules Description 12
9.3 Database Design 12
9.4 SQL Operations Used 14
9.5 Application Workflow 15
9.6 Screenshots or Outputs 16
10. Results and Discussion 20
10.1 Outputs Achieved 20
10.2 Testing 20
10.3 Discussion: Objectives Satisfied 21
10.4 Comparison with Manual Expense Tracking 21
11. Conclusion and Future Scope 22
11.1 Summary of Achievements 22
11.2 Current Limitations 22
11.3 Future Scope 22
12. References 23
13. Appendix 24

1. Course Outcomes

The following course outcomes of the Database Management System (DBMS) course are addressed by this mini project.

CO1: Design a database schema and a two-tier application for a given problem statement.

CO2: Design SQL DDL and DML queries to create, populate, maintain, and query a database using integrity constraints.

CO3: Apply database concepts in a practical web-based application.

2. PO Mapped

The project supports the following Programme Outcomes (POs).

PO1 – Engineering Knowledge: Database concepts, SQL, Python and web fundamentals were applied to build a working expense management system.

PO2 – Problem Analysis: The problems of manual expense tracking were studied and the missing features, such as category-wise reports and backups, were identified.

PO3 – Design/Development of Solutions: A database schema, application modules and web pages were designed to meet the need of recording expenses and managing budgets.

PO4 – Conduct Investigations of Complex Problems: The add, edit, delete, budget and backup functions were tested, and the outputs were checked against the expected results.

PO5 – Engineering Tool Usage: Modern tools such as MySQL Workbench, VS Code, Flask, mysql-connector-python and a web browser were used to develop and test the project.

PO8 – Individual and Collaborative Team Work: The work was shared between two team members, covering database design, application coding and documentation.

PO9 – Communication: The design and results are documented in this report, and the project can be explained through a presentation.

PO10 – Project Management and Finance: The work was planned in stages: design, implementation, testing and documentation. Only free and open-source tools were used.

PO11 – Life-Long Learning: New tools such as Flask, Jinja2 and the MySQL connector were learned during the project, which prepares the team to adopt new technologies.

3. PSO Mapped

PSO1 – Integrated Technical Proficiency: The project combines database design, SQL, Python programming and web development to implement a complete software system using modern tools.

PSO2 – Data-Driven Decision Making: The budget report uses SQL aggregate queries to compare spending with limits, which helps users make better spending decisions.

PSO3 – Interdisciplinary Collaboration and Professionalism: The project connects computing with personal finance. It was developed as a team and documented in a professional report format.

4. SDG Goals Mapped

SDG 4 – Quality Education: The project develops practical skills in database design, SQL and web application development for the students.

SDG 8 – Decent Work and Economic Growth: The skills gained in database and web development are useful for employment in the IT industry. The tool also helps users manage their money better.

SDG 9 – Industry, Innovation and Infrastructure: The project is a small technology-based solution built with a database-driven web application, which supports innovation in software development.

SDG 12 – Responsible Consumption and Production: By showing category-wise spending and budget status, the system encourages users to spend in a planned and responsible way.

5. Abstract

Managing daily spending is a basic need for every student and family, but many people still depend on notebooks or spreadsheets that are hard to organize and easy to lose. The Daily Expense Tracker is a two-tier web application developed as a DBMS mini project to solve this problem. The system is built using Python Flask for the application logic and MySQL 8 for data storage, with HTML, Jinja2 templates and basic CSS for the user interface. It allows users to record daily expenses and organize them by category using Create, Read, Update and Delete (CRUD) operations. Users can also set a monthly budget for each category and view a budget report that shows the limit, the amount spent, the remaining amount and the status. A backup feature copies expense records into a separate backup table so that the data can be viewed later. The database uses primary keys, foreign keys and unique constraints to keep the stored data correct. The main purpose of the project is to apply database design and SQL concepts to a practical and easy to use application.

6. Introduction

6.1 Background and Motivation

Keeping track of daily expenses is an important habit. It helps a person understand where money is spent and whether the spending is under control. Many people still write expenses in a notebook or enter them in a spreadsheet. These methods work for a few entries, but it becomes difficult to find totals, check each category and keep the records safe as the entries grow.

A database management system stores data in an organized way and allows it to be searched, changed and summarized using SQL. This motivated us to build a simple web application where expenses are stored in a MySQL database and viewed through a browser. The project also gave us a chance to use the concepts learned in the DBMS course, such as tables, keys, constraints and queries, in a real application.

6.2 Problem Definition

Manual methods of expense tracking do not give category-wise totals easily, do not compare spending with a monthly budget, and do not provide a proper backup of the records. The problem is to develop a simple web-based system that stores expenses in a database, organizes them by category, compares monthly spending with budget limits and keeps backup records.

6.3 Objectives

The objectives of the project are:

- To record daily expenses with date, amount, category and note.
- To maintain expense categories in the database.
- To manage monthly budgets for each category.
- To compare the actual spending with the budget limits and show the remaining amount and status.
- To store backups of the expense records in a separate table.
- To apply DBMS concepts such as schema design, constraints and SQL queries in a two-tier web application.

7. Literature Survey / Existing System

7.1 Existing Ways of Tracking Expenses

People currently track their expenses in the following ways:

- **Notebooks:** Expenses are written by hand, usually with the date and the amount.
- **Spreadsheets:** Expenses are typed into rows and columns, and totals are calculated using formulas.
- **General finance applications:** Mobile or desktop applications that record income and expenses. They often contain many features that a student may not need.

7.2 Limitations of Manual Methods

- Entries can be missed, written wrongly or lost, and there is no check on the data.
- Finding the total for one category or one month needs extra effort.
- Editing or deleting old entries can disturb the totals.
- Comparing spending with a budget has to be done separately.
- A notebook cannot be copied easily, and a spreadsheet file has no built-in backup of its own.

7.3 Gaps Identified

From the study of the existing ways, the following gaps were identified:

- Lack of simple category-wise reports.
- Lack of proper backup handling for expense records.
- Lack of a simple budget comparison that shows the limit, spent amount, remaining amount and status.
- Lack of data integrity rules, which a database provides through keys and constraints.

The proposed Daily Expense Tracker is designed to fill these gaps in a simple way.

8. Proposed System

8.1 System Overview

The Daily Expense Tracker is a two-tier web application. The first tier is the Flask web application, which handles the web pages and the application logic. The second tier is the MySQL database named expense_tracker, which stores all the data. The user works through a web browser. Every action, such as adding an expense, is sent to Flask, which runs the required SQL query on MySQL and returns the result as a web page.

8.2 Features and Scope

The following features are implemented:

- Display all expenses with date, category, amount, note, and edit and delete actions.
- Add a new expense.
- Edit an existing expense.
- Delete an expense with a confirmation.
- Monthly category-wise budget management.
- Budget report showing limit, spent amount, remaining amount and status.
- Backup of expenses into a separate backup table.
- View of all backup records.

Scope: The system is meant for personal use on a local computer. It does not include user login, charts or data export. These are listed as future scope.

8.3 System Architecture Diagram

The architecture of the system is:

User → Flask Web Application → MySQL Database

The user sends a request from the browser. The Flask application receives it, communicates with the MySQL database using mysql-connector-python, and sends the HTML page back to the user.

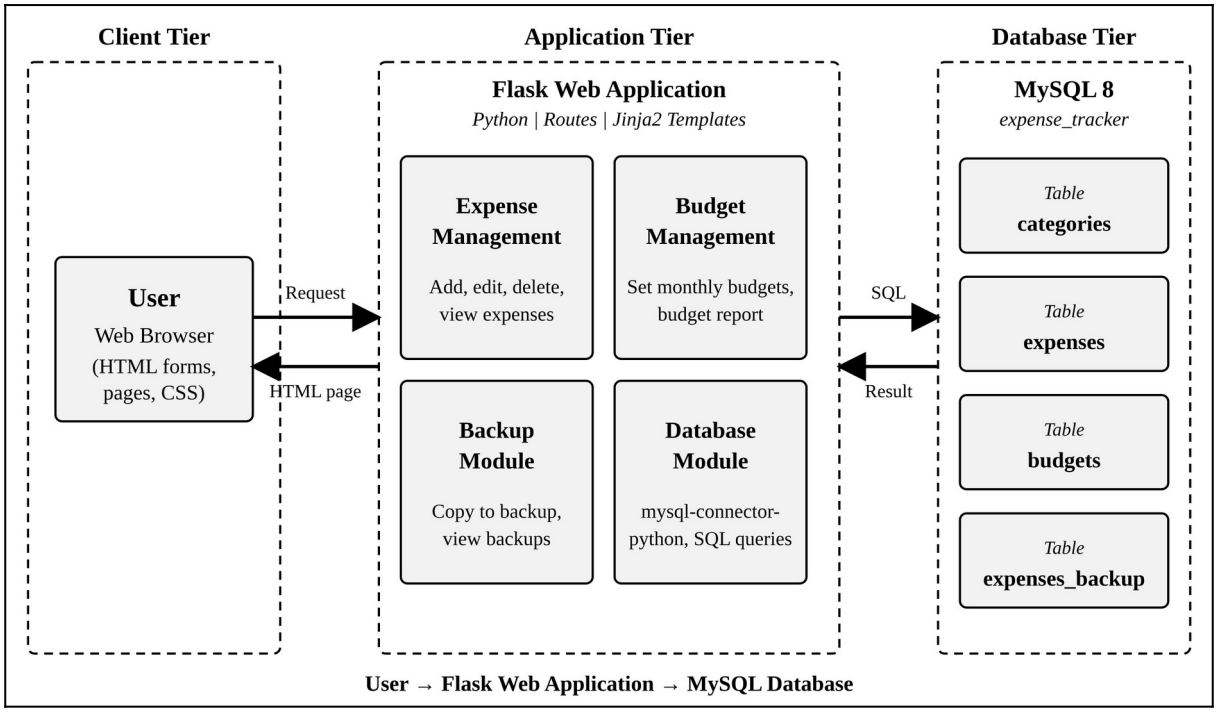


Figure 8.1: System Architecture of Daily Expense Tracker

9. Methodology / Implementation

9.1 Tools and Technologies Used

Tool / Technology	Purpose
Python Flask	Backend framework for the web application and routes
MySQL 8	Relational database to store all data (database name: expense_tracker)
mysql-connector-python	Python connector used to run SQL queries from Flask
HTML, Jinja2 templates, basic CSS	Web pages and their basic styling
MySQL Workbench	Creating and checking the tables and running SQL queries
VS Code	Code editor used to write the Python and HTML code
Web browser	Used to open and test the application

9.2 Modules Description

Expense Management Module: This module displays all expenses in a table with date, category, amount and note. It lets the user add a new expense, edit an existing expense and delete an expense. Before deleting, the system asks for a confirmation so that a record is not removed by mistake.

Budget Management Module: This module lets the user set a monthly budget limit for each category. The budget report then shows the limit, the amount spent, the remaining amount and the status for each category by comparing the budget with the total of the expenses.

Backup Module: This module copies the records of the expenses table into the separate expenses_backup table. Each backup record also stores the date and time of the backup in the backed_up_at column. The user can view all backup records on a separate page.

Database Module: This module contains the connection to the MySQL database and all the SQL queries used by the other modules. It manages the four tables: categories, expenses, budgets and expenses_backup.

9.3 Database Design

The database expense_tracker has four tables. Their columns are described below.

Table 1: categories

Column	Description	Key / Constraint
category_id	Unique number for each category	Primary key
name	Name of the category	Unique

Table 2: expenses

Column	Description	Key / Constraint
expense_id	Unique number for each expense	Primary key
amount	Amount spent	Required
category_id	Category of the expense	Foreign key to categories
expense_date	Date of the expense	Required
note	Short note about the expense	Optional

Table 3: budgets

Column	Description	Key / Constraint
budget_id	Unique number for each budget	Primary key
category_id	Category for which the budget is set	Foreign key to categories
month_year	Month and year of the budget	Unique with category_id
limit_amount	Budget limit for the month	Required

Table 4: expenses_backup

Column	Description	Key / Constraint
expense_id	Expense number copied from expenses	—
amount	Amount copied from expenses	—
category_id	Category copied from expenses	—
expense_date	Date copied from expenses	—
note	Note copied from expenses	—
backed_up_at	Date and time when the backup was taken	Set at backup time

Relationships: One category can have many expenses, and one category can have many budgets (one for each month). The expenses and budgets tables are linked to the categories table through the category_id foreign key. The expenses_backup table stores copies of expense records.

The Entity-Relationship (ER) diagram of the system is shown in Figure 9.1.

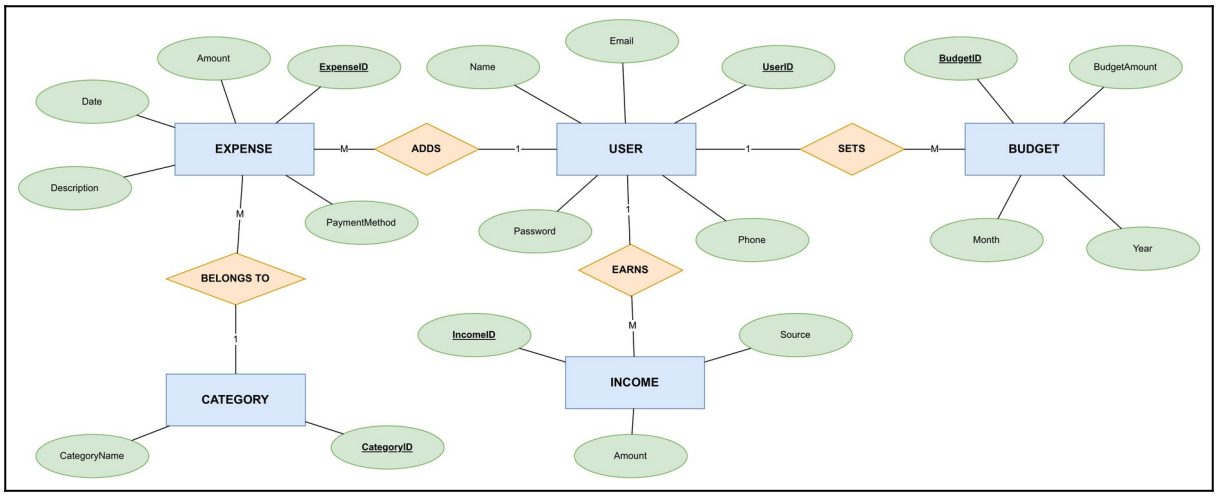


Figure 9.1: ER Diagram of Daily Expense Tracker

9.4 SQL Operations Used

The project uses the following SQL features:

SQL feature	**Use in the project**
CREATE TABLE	Creating the four tables of the database
INSERT	Adding a new expense, category or budget
SELECT	Displaying expenses, budgets, reports and backup records
UPDATE	Editing an existing expense
DELETE	Deleting an expense
JOIN	Showing category names with expenses and budgets
GROUP BY and aggregate functions	Finding the total spent for each category (SUM)
Foreign keys	Linking expenses and budgets to categories
Unique constraints	Avoiding duplicate categories and duplicate budgets
Backup insertion query	Copying expense records into expenses_backup

An example of a query used for the budget report is shown below. It joins the tables, groups the data by category and uses the SUM function.

```
SELECT c.name, b.limit_amount,
       COALESCE(SUM(e.amount), 0) AS spent_amount,
       b.limit_amount - COALESCE(SUM(e.amount), 0) AS remaining_amount
FROM budgets b
JOIN categories c ON b.category_id = c.category_id
LEFT JOIN expenses e ON e.category_id = b.category_id
     AND DATE_FORMAT(e.expense_date, '%Y-%m') = b.month_year
WHERE b.month_year = '2026-10'
GROUP BY b.budget_id, c.name, b.limit_amount;
```

9.5 Application Workflow

The working of the application is shown in the following flow:

Start → Open Web Application → Add or Manage Expense → Store Data in MySQL → View Expense and Budget Reports → Create Backup

The user starts the Flask application and opens it in the browser. The user then adds a new expense or manages an existing one. The data is stored in the MySQL database. After this, the user can view the list of expenses and the budget report. Finally, the user can create a backup of the expenses and view the backup records.

9.6 Screenshots or Outputs

The home page lists all expenses with date, category, amount and note. Each row has an Edit link and a Delete button.

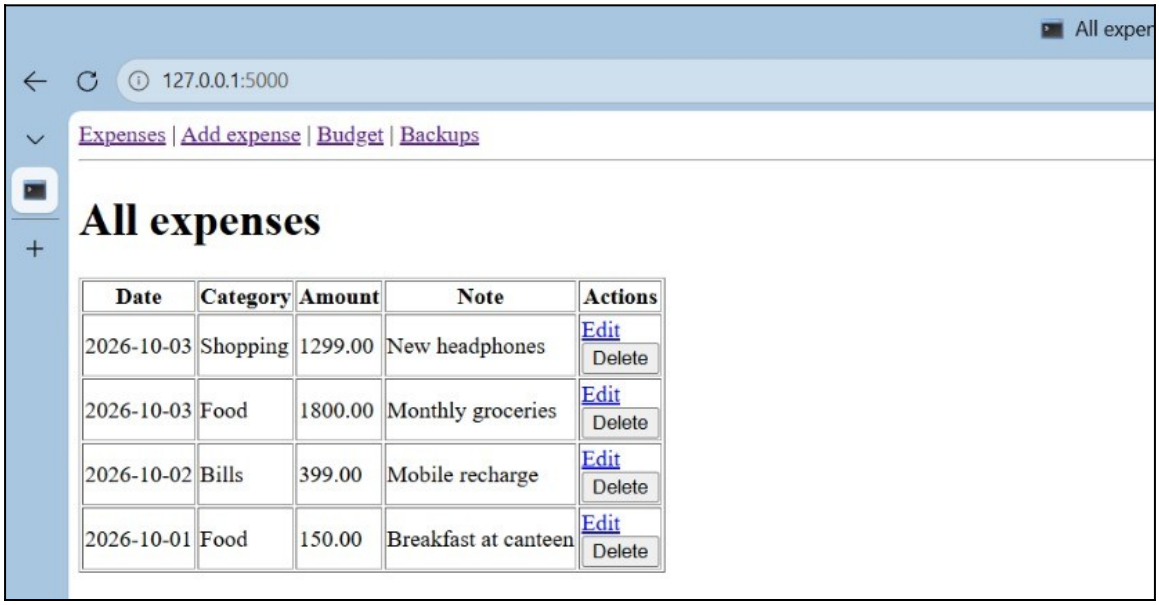


Figure 9.1: Home Page

The Add expense page has a form to enter the amount, category, date and note.

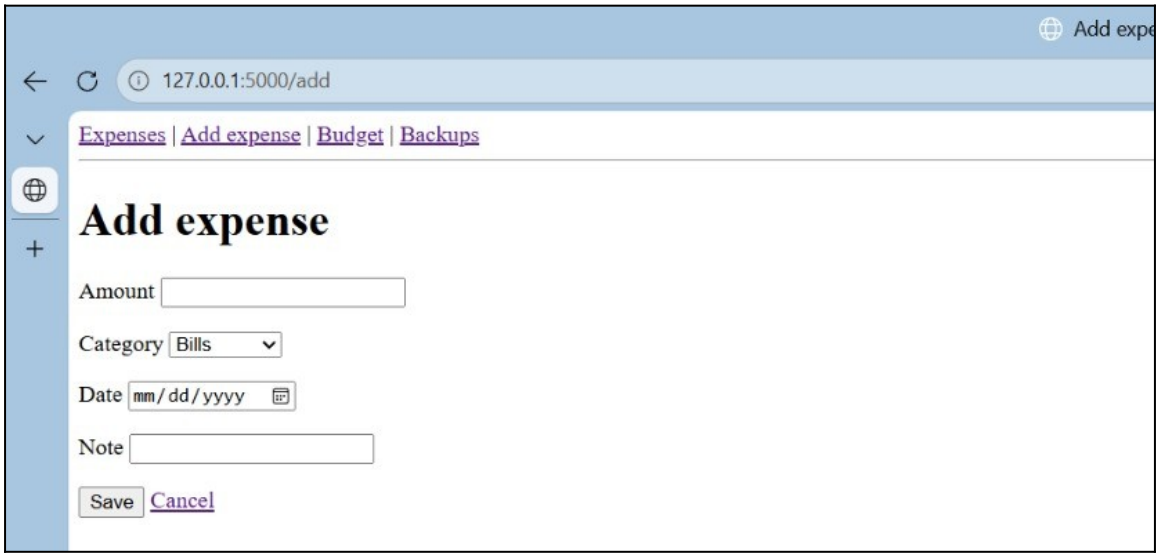


Figure 9.2: Add Expense Page

The Edit expense page shows the saved values of a record so that they can be changed and saved.

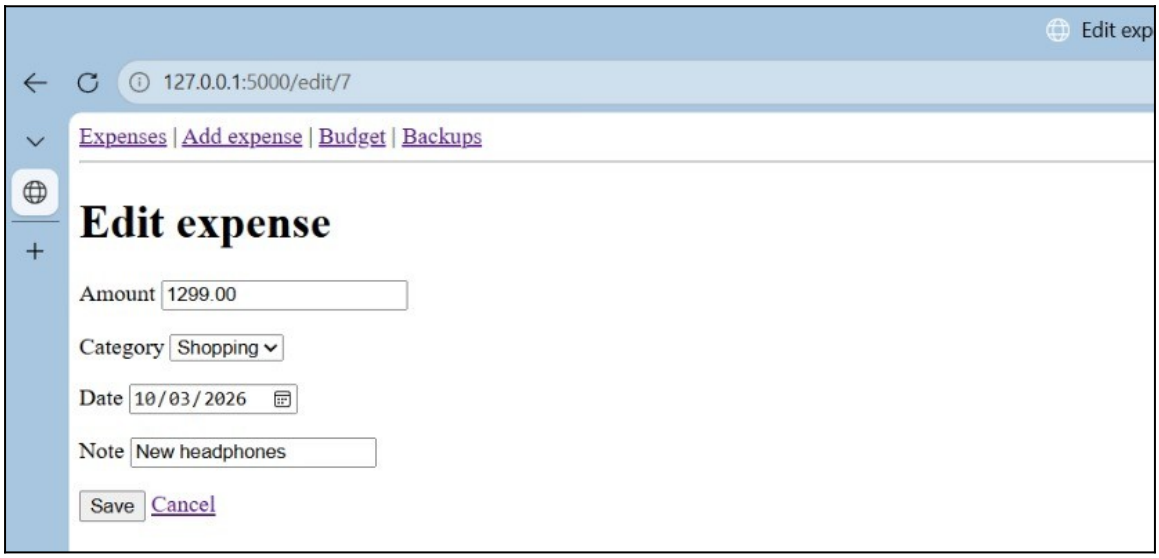


Figure 9.3: Edit Expense Page

The Budget page is used to set a monthly limit for a category. The budget report below the form shows the limit, spent amount, remaining amount and status.

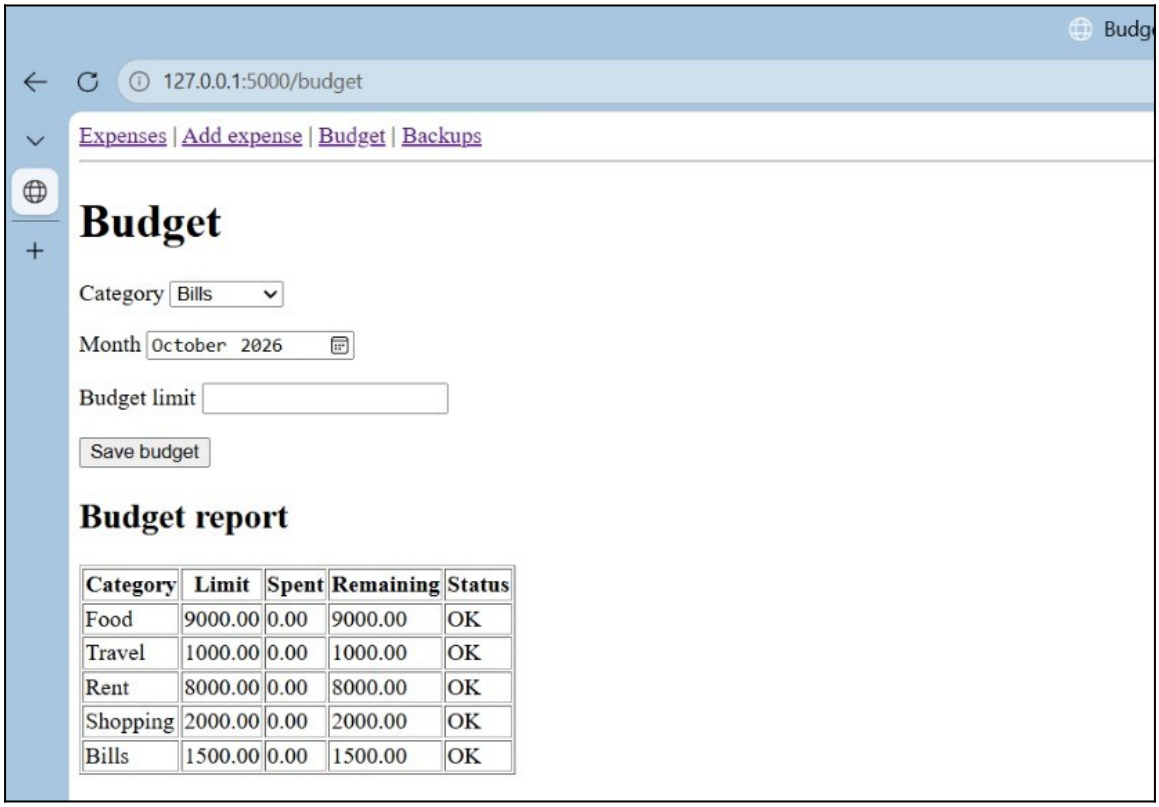
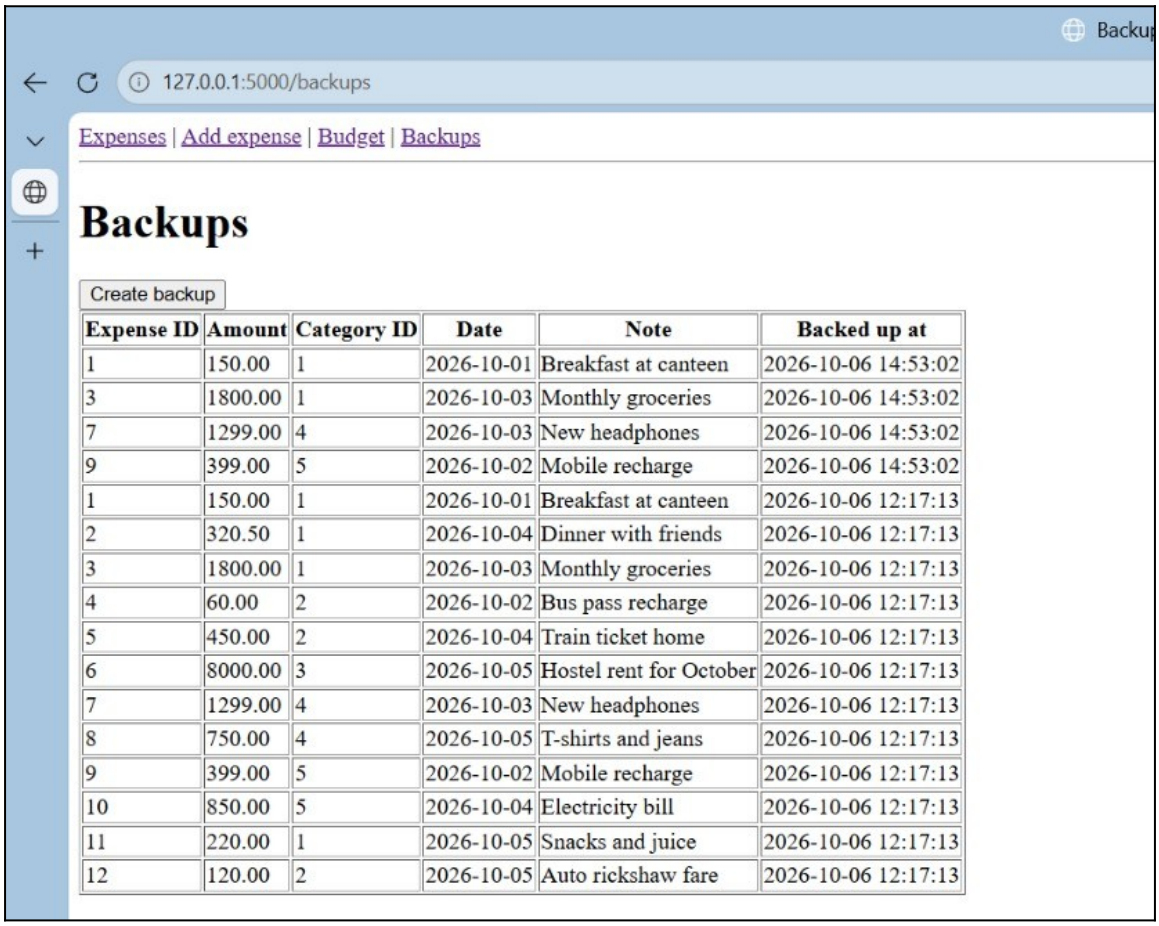


Figure 9.4: Budget Page

The Backups page has a Create backup button and lists all backup records with the time of backup.



Expense ID	Amount	Category ID	Date	Note	Backed up at
1	150.00	1	2026-10-01	Breakfast at canteen	2026-10-06 14:53:02
3	1800.00	1	2026-10-03	Monthly groceries	2026-10-06 14:53:02
7	1299.00	4	2026-10-03	New headphones	2026-10-06 14:53:02
9	399.00	5	2026-10-02	Mobile recharge	2026-10-06 14:53:02
1	150.00	1	2026-10-01	Breakfast at canteen	2026-10-06 12:17:13
2	320.50	1	2026-10-04	Dinner with friends	2026-10-06 12:17:13
3	1800.00	1	2026-10-03	Monthly groceries	2026-10-06 12:17:13
4	60.00	2	2026-10-02	Bus pass recharge	2026-10-06 12:17:13
5	450.00	2	2026-10-04	Train ticket home	2026-10-06 12:17:13
6	8000.00	3	2026-10-05	Hostel rent for October	2026-10-06 12:17:13
7	1299.00	4	2026-10-03	New headphones	2026-10-06 12:17:13
8	750.00	4	2026-10-05	T-shirts and jeans	2026-10-06 12:17:13
9	399.00	5	2026-10-02	Mobile recharge	2026-10-06 12:17:13
10	850.00	5	2026-10-04	Electricity bill	2026-10-06 12:17:13
11	220.00	1	2026-10-05	Snacks and juice	2026-10-06 12:17:13
12	120.00	2	2026-10-05	Auto rickshaw fare	2026-10-06 12:17:13

Figure 9.5: Backups Page

10. Results and Discussion

10.1 Outputs Achieved

- The home page displays all expenses with date, category, amount and note, along with edit and delete actions.
- New expenses can be added and existing expenses can be edited through web forms.
- An expense is deleted only after the user confirms the action.
- Monthly budgets can be set by choosing a category, a month and a budget limit, and the budget report shows the limit, spent amount, remaining amount and status (for example, OK).
- Expenses can be backed up with the Create backup button, and all backup records are listed with their backup date and time.
- Every page has a navigation bar with the links Expenses, Add expense, Budget and Backups.

10.2 Testing

Each main function was tested by performing the action in the browser and checking the result on the web page and in MySQL Workbench.

Function	Test performed	Expected result	Result
Add expense	Enter date, category, amount and note and submit	New record appears in the expense list and in the expenses table	Successful
Edit expense	Change the amount or note of a record and save	Updated values appear in the list and in the table	Successful
Delete expense	Click delete and confirm	Record is removed only after confirmation	Successful
Budget	Set a monthly limit for a category and open the budget report	Report shows limit, spent, remaining and status	Successful
Backup	Create a backup and open the backups page	Expense records are copied to expenses_backup and listed	Successful

10.3 Discussion: Objectives Satisfied

- **Recording expenses:** achieved through the add, edit and delete functions.
- **Maintaining categories:** every expense and budget is linked to a category stored in the categories table.
- **Managing budgets:** monthly category-wise limits can be set and stored in the budgets table.
- **Comparing spending with budget limits:** the budget report shows the limit, spent amount, remaining amount and status.
- **Storing backups:** expenses are copied to a separate expenses_backup table and can be viewed.

10.4 Comparison with Manual Expense Tracking

Point	Manual tracking	Daily Expense Tracker
Recording	Written by hand or typed in a sheet	Entered through web forms and stored in MySQL
Category totals	Calculated by hand	Calculated using SQL (GROUP BY and SUM)
Budget comparison	Done separately	Budget report shows limit, spent, remaining and status
Edit and delete	Can disturb the records	Done through the application, with delete confirmation
Data correctness	No rules on the data	Keys and constraints in the database
Backup	Not available or manual	Backup table with backed_up_at

11. Conclusion and Future Scope

11.1 Summary of Achievements

The Daily Expense Tracker was successfully developed using Flask and MySQL. The system can record and manage expenses, organize them by category, manage monthly budgets, show a budget report and keep backup records in a separate table. The project helped us apply DBMS concepts such as table design, primary and foreign keys, unique constraints, joins and aggregate queries in a practical web application.

11.2 Current Limitations

- The application can be used only on a local computer (local-only access).
- There is no user login, so all data is available to anyone who opens the application.
- Reports are shown only as tables; there are no charts.
- Data cannot be exported to other file formats.

11.3 Future Scope

- User authentication with login and registration.
- Charts and graphs for expenses and budgets.
- Export of reports to PDF or Excel.
- Search and filter options for expenses.
- Mobile-friendly design.
- Online deployment so that the system can be used from anywhere.

12. References

[1] Pallets Projects, "Flask Documentation." [Online]. Available: https://flask.palletsprojects.com/

[2] Oracle Corporation, "MySQL Documentation." [Online]. Available: https://dev.mysql.com/doc/

[3] Python Software Foundation, "Python Documentation." [Online]. Available: https://docs.python.org/3/

[4] Oracle Corporation, "MySQL Connector/Python Documentation." [Online]. Available: https://dev.mysql.com/doc/connector-python/en/

13. Appendix

Short code snippets from the project are given below.

A. MySQL Connection Code

```
import mysql.connector

def get_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="your_password",
        database="expense_tracker"
    )
```

B. CREATE TABLE Statement

```
CREATE TABLE expenses (
    expense_id INT AUTO_INCREMENT PRIMARY KEY,
    amount DECIMAL(10,2) NOT NULL,
    category_id INT NOT NULL,
    expense_date DATE NOT NULL,
    note VARCHAR(255),
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
```

C. INSERT Query

```
INSERT INTO expenses (amount, category_id, expense_date, note)
VALUES (250.00, 1, '2026-10-05', 'Lunch');
```

D. Flask Route (Add Expense)

```
@app.route("/add", methods=["GET", "POST"])
def add_expense():
    conn = get_connection()
    cursor = conn.cursor()
    if request.method == "POST":
        cursor.execute(
            "INSERT INTO expenses (amount, category_id, expense_date, note) "
            "VALUES (%s, %s, %s, %s)",
            (request.form["amount"], request.form["category_id"],
             request.form["expense_date"], request.form["note"]))
        conn.commit()
        return redirect(url_for("index"))
    cursor.execute("SELECT category_id, name FROM categories")
    categories = cursor.fetchall()
    return render_template("add_expense.html", categories=categories)
```

E. HTML Form Snippet

```
<form method="POST">
  <label>Amount</label>
  <input type="number" step="0.01" name="amount" required>
  <label>Category</label>
  <select name="category_id">
    {% for c in categories %}
    <option value="{{ c[0] }}">{{ c[1] }}</option>
    {% endfor %}
  </select>
  <label>Date</label>
  <input type="date" name="expense_date" required>
  <label>Note</label>
  <input type="text" name="note">
  <button type="submit">Add Expense</button>
</form>
```